

班级 2116029

学号 21009100863

本科毕业设计（论文）

外文资料翻译

毕业设计题目	RISCV 开源 IP 平台对于移植开源飞控的资
	源 匹 配 性 研 究

外文资料题目	RISCV 是否准备好进入太空？从安全角度
	探 讨

学 院	空间科学与技术学院
-----	-----------

专 业	空间科学与技术
-----	---------

学 生 姓 名	何成楷
---------	-----

指导教师姓名	韩高
--------	----

摘要

用于空间应用的集成电路通常具有低产量和高性能要求。因此，采用商用现货（COTS）组件和第三方知识产权内核（3PIPs）对于使系统设计、实现和部署在性能方面具有成本效益和可行性至关重要。然而，这种设计范式在设计阶段和运行阶段都会使系统面临多种安全威胁。在本文中，我们主要讨论与空间应用相关的安全问题，重点关注由于采用知名的 RISC-V 微处理器而带来的威胁。我们强调，硬件特洛伊木马（HTHs）和微架构侧信道攻击（MSCAs）可能会通过改变系统的正常行为或窃取机密信息来危及整个系统的运行。我们还讨论了 RISC-V 架构提供的安全扩展及其局限性。文章最后对 RISC-V 微处理器在空间领域安全性方面仍待解决的问题进行了概述。

关键词： 微架构侧信道攻击、微处理器、硬件安全、硬件特洛伊木马、RISC-V、空间应用。

第一章 引言

在空间系统中使用商用现货（COTS）组件、第三方知识产权内核（3PIPs）和商业计算机辅助设计（CAD）工具的想法正变得越来越流行。事实上，这种设计选择在集成电路产量极低的应用场景（如空间应用）中，代表了性能与成本之间的一个有趣的权衡。其基本思想是在空间系统中重用遗留的（即非专门为太空设计的）子系统、组件和 3PIPs，以同时受益于降低的开发成本和高性能。

当专注于微处理器时，设计者可以选择定义自己的指令集架构（ISA），或者使用已有的 ISA。尽管使用开源软件可以被视为一种遗留程序，但硬件尚未完全经历开放性带来的颠覆性影响。然而，在过去几年中，RISC-V 架构的流行度迅速上升，吸引了许多之前专注于其他开放和免费 ISA、专有 ISA，甚至是自行设计 ISA（其主要缺点是需要设计和维护一个软件生态系统）的大学和公司的关注。尽管这种设计范式可以降低成本，但它也使系统在设计阶段和运行阶段面临多种安全威胁。商业集成电路（ICs）的生产特点是全球化的供应链。这种全球化的供应链带来的好处是降低了设计成本和时间，但另一方面，这也导致了对最终 ICs 的信任度降低。其后果是很难确保/评估供应链中所有参与方的可信度。因此，产品面临多种威胁：晶圆厂可能会超量生产 ICs 并将其出售到黑市；有缺陷或被废弃的 ICs 可能会被当作合格品交付；IP 内核许可可能会被侵犯，IP 内核可能会被过度使用；设计可能会被恶意修改，以在最终产品中插入隐蔽的不受欢迎的功能，即硬件特洛伊木马（HTHs）。

微处理器系统面临的一种新型威胁是微架构攻击。这些攻击是侧信道攻击（SCA）的一个子类，即通过分析时序信息、功耗、热足迹或电磁辐射等信息来泄露系统中未经授权的信息的攻击。另一方面，微架构攻击不需要攻击者对被攻击系统进行物理访问。事实上，这种攻击仅依赖于对系统运行敏感应用程序时的时序行为的观察。这一类攻击的基本思想是，由于计算机架构针对处理速度进行了优化，因此处理数据、内存访问和执行时间之间存在很强的相关性：如果被攻击进程和攻击者进程共享相同的缓存空间，这种相关性可能成为一个可利用的侧信道。一个著名的微架构攻击案例是熔断（Meltdown），攻击者利用乱序执行破坏地址空间隔离，而无需利用任何软件漏洞。因此，通过利用熔断，攻击者允许自己的程序访问其他程序以及操作系统的内存（从而也可以访问秘密信息）。

在过去，硬件安全并未被航天工业视为问题，因为大多数攻击（例如故障攻击）都需要攻击者对被攻击系统进行物理访问，而航天应用显然不需要。然而，基于 MSCAs 和 HTHs 的攻击可以在没有对系统进行物理访问（在攻击发生时）的情况下成功实施。事实上，一旦植入，HTH 可以通过执行触发程序远程激活，同时 MSCA 可以通过强制执行与被攻击程序并行运行的恶意软件片段来实施。因此，如今在考虑用于航天应用的数字设计时，也应该解决硬件安全问题。

微处理器存在于星载数据处理系统和星载数据处理系统中，既有 ASICs/SoCs 中的硬核，也有 FPGAs 中的软核。微处理器是运行姿态确定与控制系统（ADCS）所需算法以及执行地面控制或基于算法的指令与数据处理（C&DH）的关键组件。方向对于卫星的任务目标至关重要（例如，错误的方向将使与地面站的通信变得不可能）。C&DH 的执行也至关重要，因为地面站与卫星之间存在主/代理关系。

在这篇前瞻性论文中，我们讨论了在空间应用中采用 RISC-V 平台的安全问题。我们首先介绍 RISC-V 架构本身提供的安全特性，并介绍其在防止和减轻硬件安全相关攻击的影响方面的局限性。事实上，专注于硬件安全，我们强调硬件特洛伊木马（HTHs）和软件威胁（特别是微架构侧信道攻击（MSCAs））如何通过改变其正常行为或窃取秘密信息来破坏整个系统的功能。最后，我们概述了来自研究的现有解决方案，旨在进一步确保 RISC-V 处理器的采用，并提出并讨论尚未解决的问题。本文旨在帮助技术人员找到最佳架构配置，以满足性能和安全要求。同时，这项工作也可以作为年轻研究人员和学生的起点，通过提供保护 RISC-V 的最新解决方案的讨论以及对未解决问题的概述。

本文的其余部分安排如下：第二部分描述空间系统中可能的威胁及其对不同航天任务的影响；第三部分讨论 RISC-V 架构提供的主要安全相关特性及其局限性；最后，第四部分总结全文，强调未解决的问题。

第二章 空间数据系统中的安全性

在空间嵌入式系统领域，我们正在见证一种新的趋势。尽管“传统”的处理 ASICs 仍然受到“地球”处理器与空间级处理器之间通常不断扩大的性能差距的困扰（如[19]中所讨论的），然而，最近在基于 SRAM 的 FPGA 方面的进展已经将地球上的“前沿”技术引入空间应用，为空间处理提供了可能的飞跃。特别是在有效载荷数据处理任务中，这些设备很可能会取代任何专用的应用特定标准产品（ASSP），并要求为特定应用提供复杂的处理器 IP 内核。

接下来，我们回顾咨询委员会为空间数据系统（CCSDS）在[20]中强调的主要与空间任务相关的安全问题。

1) 有意数据篡改

攻击者可能会在其源头故意篡改数据（例如，通过干扰传感器），或者由靠近被攻击卫星的攻击卫星进行操作[18]。另一种可能性是，在数据从仪器传输到地面系统的过程中，数据被篡改，例如通过执行恶意软件。除了仪器信息的损坏外，故意的数据篡改可能会导致系统的灾难性损失，例如，如果在接收到命令时，航天器没有采取行动或采取了错误的行动。例如，如果导航机动命令被篡改，航天器最终可能会进入不可用轨道，错过与目标的相遇，甚至被摧毁。

2) 软件威胁

编码错误可能会发生并导致安全问题[20]。此外，系统操作员可能会错误配置系统，从而导致安全漏洞。另一方面，程序员可能会引入逻辑或实现错误，从而导致系统漏洞。最后，外部实体可能会试图利用漏洞注入软件或更改配置。其结果可能是数据丢失、航天器失控、未经授权的航天器控制或任务失败。

软件威胁（STs）也可能来自硬件和架构级别的漏洞利用。例如，高性能处理器可能会采用推测执行和乱序执行。这些特性可能会被恶意利用，以访问受保护的处理器内存位置，从而为所谓的微架构侧信道攻击（MSCAs）[12]打开大门。这种类型的 SCA 特别关键（对于空间应用也是如此），因为它不需要对被攻击系统进行物理访问，而是依赖于对攻击本身感兴趣的特征的监测，例如，被攻击处理器的时序行为、性能计数器等。一些 MSCA 的例子包括熔断（Meltdown）[17]、幽灵（Spectre）[21]和预兆（Foreshadow）[22]，攻击者利用乱序和推测执行破坏地址空间隔离，而无需利用任何软件漏洞。因此，这些攻击允许攻击者访

问未经授权的内存区域并窃取信息。结果是，使用可信组件构建的系统可能会被成功攻击。

3) 恶意硬件

硬件信任问题源于设计系统生命周期中涉及的不可信实体，包括不可信的 IP 提供商、设计团队组件、CAD 工具以及制造、测试和分发设施[23]。当使用 COTS 组件时，恶意硬件可能会进入系统。这些组件通常没有可信的供应链，并被部署到大众市场。事实上，COTS 组件的供应链由许多（可能不可信的）实体组成[23]。

为了进行恶意活动而对现有电路元件进行的不希望的添加或修改被称为硬件特洛伊木马（HT）。HT 可以改变电路的功能、降低其可靠性或泄露有价值的信息[23]。已知软件可利用的 HT 可能会被集成到微处理器中。借助 HT，攻击者可能能够执行自己的恶意软件、修改正在运行的软件或获取 root 权限[24]、[25]、[26]；或者他们可能能够通过软件激活的后门进入特权模式[27]。最后，必须提到的是，最新出现的攻击向量是所使用的 CAD 工具。事实上，已经证明 HTHs 可以通过用于高级和逻辑综合、布局布线以及比特流生成的工具被引入到设计电路中[28]、[29]。

安全相关威胁的影响高度依赖于任务轨道和系统所执行的任务类型。接下来，我们讨论这两点。

1) 任务轨道

一方面，地球静止轨道（GEO）卫星比低地球轨道（LEO）任务更安全，因为它们需要具有更高发射功率和更大天线的地面站。当然，这使得与卫星的通信更加昂贵，从而限制了可能的攻击者数量。另一方面，GEO 卫星比 LEO 卫星更容易受到攻击，因为它们在一定地理区域持续可见，而 LEO 卫星仅在特定时间段内从地面站可见。此外，卫星星座增加了整个空间系统的脆弱性，因为在这种情况下，轨道上有许多卫星，从而增加了从地球上特定点的可见窗口。最后，深空/行星际任务需要比 GEO 卫星更大的天线和更高的功率来进行攻击[20]。

2) 任务类型

空间系统所执行的任务类型以及其安全关键性或任务关键性，高度影响安全相关威胁的影响和可能性。接下来，我们讨论几种类型的空间任务及其相关的安全问题。

地球观测卫星：这些卫星可以是科学用途的，也可以是政府的关键资产（例如，气象预测应用）。在后一种情况下，最可能的威胁是未经授权的访问，通常通过加密命令和任务数据以及认证命令来应对。此外，恶意硬件是可能的威胁，可以通过分析硬件功能或谨慎选择可信供应商来缓解。数据损坏也可能是一个问题，向最终用户提供错误信息。

通信卫星：这些卫星旨在提供通常具有非常高可用性的服务。例如，在 GEO 通信卫星的情况下，整个空间系统预计 99.9% 的时间内提供服务[30]。因此，这种类型的卫星特别容易受到攻击：事实上，即使可用性略有降低，也可能对服务产生巨大影响。对通信卫星的攻击案例在[18]中有所报道：2003 年，通过一个上行站向 Telesat 的 12 颗卫星发送矛盾的频率，覆盖了信号，有效地阻断了电视节目。

导航卫星：这些通常是双用途卫星，通常属于政府的关键基础设施。一个例子是 GPS，它被个人、公司和军事机构使用。与通信卫星一样，导航卫星系统的丢失可能导致潜在的生命损失、个人安全和关键基础设施的损失。

科学任务：这些任务通常不属于关键基础设施的一部分。尽管它们也暴露于与其他任务相同的威胁，但影响要小得多，对任务本身所涉及的实体的财务投资和声誉造成损害[20]。然而，投资可能非常高：例如，最近发射的詹姆斯·韦伯太空望远镜的成本达到了 100 亿美元[31]。

载人任务：从安全角度来看，这些任务当然是最关键的。事实上，在这种情况下，攻击导致航天器丢失的后果可能是灾难性的，可能会对机组人员的生命造成威胁。

第三章 RISC-V 的安全性考量

在本节中，我们介绍 RISC-V 架构，并讨论其现有的与安全相关的特性；最后，我们对现有的局限性以及一些提议的安全扩展进行综述。

A. RISC-V 指令集架构

RISC-V 最初由加州大学伯克利分校开发，旨在支持面向硬件实现的计算机架构研究和教育[32]。这种开放且免费的指令集架构（ISA）的传播，已经为地球应用（例如安全、人工智能等）开辟了广阔的科研领域。事实上，访问专有架构成本高昂，且限制了对某些产品或特定科研活动的使用范围。一个由大型开源社区支持的软件生态系统，激发了前所未有的开发活动，许多开源实现被宣布和

/或发布。因此，采用一种流行的免费和开放的 ISA 可以通过重用实现更快的上市时间和更低的成本。

RISC-V ISA 规范的主要特点是其模块化，这使得它能够覆盖广泛的应用场景，并增加软件的可重用性（通过将新指令作为可选扩展添加，而不是发布整个 ISA 的新版本）。RISC-V 允许使用标准和非标准扩展（在规范之外定义）。RISC-V 的用户级 ISA 定义为一个基础整数（I）ISA，它必须出现在任何实现中，以及对基础 ISA 的可选扩展。当实现针对小型 32 位微控制器时，可以选择实现基础整数的一个子集（E），使用 16 个通用寄存器而不是 32 个。标准定义了一个“通用”子集（G）以及通用计算系统所需的扩展集合。

B. 内存隔离

在软件层面保护系统安全的主要特性是内存隔离（即每个进程都有其自己的地址空间，从而在程序之间隔离内存）[33]。隔离意味着即使一个分区的安全性被破坏，攻击者也无法侵犯其他分区的安全性。Linux 内核提供了内核服务与用户空间应用程序之间的分离。然而，这种方式使得内核在安全性方面成为单点故障，因此它可能负责大量漏洞，鉴于其巨大的攻击面[34]。

内存隔离可以通过使用虚拟机管理程序（hypervisors）来进一步改进，以在多核处理器上运行的不同操作系统实例之间提供隔离。目前，内存隔离正在被研究用于空间应用，以改进安全行为，即避免一个操作系统上运行的应用程序的故障可能影响在同一处理器上另一个操作系统中运行的另一个应用程序的执行[35]。鉴于软件系统的复杂性不断增加，即使在空间应用中，设计者也越来越多地依赖第三方软件组件，通常以软件库的形式提供。当这些库的代码是开源的时，可以进行安全分析和验证。另一方面，专有软件通常以可链接的（且不可检查的）对象模块形式提供。因此，通常需要在系统内强制分离各种软件组件[34]。

C. RISC-V 软件栈和特权级别

RISC-V 的模块化特性允许实现不同安全级别的不同软件栈。例如，图 1 展示了在 RISC-V 特权规范[36]中描述的软件栈。在图的左侧，展示了一个简单的软件栈实现。应用程序通过应用程序二进制接口（ABI）与应用程序执行环境（AEE）进行交互。在这种简单情况下，ABI 由实现的用户级 ISA 子集组成。在图的中央，多个应用程序运行并通过 ABI 与操作系统（OS）通信（后者在此案例中提供 AEE）。反过来，操作系统通过一个监督二进制接口（SBI）与一个

监督执行环境（SEE）通信。SBI 包括用户级和监督级 ISA，以及一组 SBI 函数调用。SEE 可以是一个低端硬件平台上的简单引导程序和 BIOS 风格的输入/输出系统，也可以是一个高端服务器中的虚拟机，或者是一个架构仿真环境中的主机操作系统的薄翻译层。在图的右侧，RISC-V 运行一个虚拟机监视器配置，其中多个操作系统运行在虚拟机管理程序之上。每个操作系统通过 SBI 与虚拟机管理程序通信，后者反过来提供 SEE。虚拟机管理程序通过一个虚拟机管理程序二进制接口（HBI）与虚拟机管理程序执行环境（HEE）通信，以将虚拟机管理程序与硬件平台隔离。

特权级别用于在上述软件栈的不同组件之间提供保护。线程可能具有的可能级别包括：

机器模式（M 模式）：具有最高特权级别，且是唯一强制性的模式。运行在机器模式下的代码必须完全可信，因为它可以访问机器实现。

监督模式：被添加以在操作系统和 SEE 之间提供隔离。

用户模式：专用于用户应用程序。

特权级别被设计和检查，以便任何尝试执行当前模式未允许的操作的尝试都将引发异常。任何特定的 RISC-V 实现都可以选择实现哪些模式（除了强制性的 M 模式）。这允许硬件架构师在提供的安全级别和实现成本之间找到合适的权衡[36]。典型解决方案包括：

M：简单嵌入式系统，安全性不是问题。这种解决方案不会对错误或恶意应用程序代码提供任何保护。

M、U：安全嵌入式系统。

M、S、U：运行类 Unix 操作系统的系统。添加监督模式以在操作系统和 SEE 之间提供隔离。

用户线程通常在用户模式下运行应用程序代码，直到陷阱（例如，监督调用或定时器中断）强制切换到陷阱处理程序，该处理程序通常在更高级别的模式下运行。陷阱处理程序将被执行，最终用户线程将在原始陷阱指令处或之后恢复执行，再次处于用户模式。

除了不同的特权模式外，RISC-V ISA 还提供了额外的扩展以增强安全性。

1) 物理内存保护

M 模式支持一个可选的标准扩展，用于物理内存保护（PMP）。PMP 扩展定义

了控制寄存器，用于指定每个物理内存区域的访问权限（读、写、执行）。尝试从没有执行权限的 PMP 区域获取指令，或者从没有读权限的物理内存位置加载数据，将引发异常。

使用 PMP 提供了许多安全特性。例如，线程被阻止修改甚至读取共享库的数据。因此，可以使用 PMP 将共享库数据所在的内存区域设置为仅执行。即使 PMP 主要用于防止低特权级别（例如 U 和 S 模式）运行的线程访问特权内存内容，“锁定”功能即使仅实现 M 模式时也能提供保护。事实上，如果锁定 PMP 条目，则对该条目的配置寄存器和相关地址寄存器的写操作将被忽略。实际上，PMP 限制对 M 模式也有效：锁定的 PMP 条目只能通过系统复位解锁。

2) 密码学扩展

航天器安全的基本要求是使用 AES 加密标准对遥测和遥控信号进行加密[37]。密码学扩展的实现提高了性能，并将实现细节隐藏在软件之外，从而减少了攻击面[38]。然而，通常更倾向于在单独的硬件组件或板卡上实现遥测和指令的加密和解密，因为要求密钥不能从软件访问。

3) 用户级中断扩展 (N)

此扩展允许具有不同特权级别的中断处理程序之间实现隔离。事实上，中断处理程序可以在用户级别执行，因此无法破坏隔离模型[34]。

大多数针对隔离的攻击利用对处理器的物理访问来执行故障注入 (FI) [39]。在空间应用中，这可能比移动和物联网 (IoT) 等应用更复杂，尽管从原则上仍然可能。例如，在[39]中展示了如何利用每条指令的功耗知识，在上下文切换期间跳过一条指令，从而允许访问受 PMP 保护的内存部分。另一方面，前面提到的攻击（即 MSCAs 和 HTHs）可以在攻击者在攻击时没有任何对被攻击系统的物理访问的情况下成功实施。事实上，HTHs 可以在设计阶段由参与设计的许多不可信方之一植入。一旦植入，HTH 可以通过执行触发程序“轻松”激活。同样，MSCA 是与被攻击软件并行运行的恶意软件。因此，MSCAs 和 HTHs 也必须被视为空间场景中的严重威胁，因为在空间场景中，攻击者几乎不可能对系统进行物理访问。

尽管 RISC-V 架构中集成的原生安全特性无法保护系统免受 MSCAs 和 HTHs 的攻击，但我们认为应该将来自研究的创新安全解决方案集成到 RISC-V 架构中，以使采用这种微处理器的空间任务安全且可信。

最近，针对基于英特尔和 ARM 处理器的系统，提出了一些旨在抵御 MSCAs 和 HTHs 的系统级方法。特别是，新趋势旨在在由不可信组件构建的系统上提供安全且可信的程序执行。这些方法背后的大部分思想是引入硬件安全检查器（HSCs），能够监控取指活动、处理的数据

第四章 讨论与待解决的问题

随着在空间应用中部署商用微处理器的兴趣不断增加，以及新型硬件导向威胁（例如微架构侧信道攻击（MSCAs）和硬件特洛伊木马（HTHs））的出现，安全性（尤其是硬件安全性）成为一个开放且严重的问题。

正如我们在本文中所讨论的，RISC-V 微处理器已经实现了许多安全特性。事实上，物理内存保护（PMP）、密码学扩展和用户级中断等特性已经能够保护基于 RISC-V 的系统免受多种攻击，例如数据篡改和未经授权的访问。然而，RISC-V 处理器对 MSCAs 和 HTHs 的防护仍然是一个亟待解决的问题。

除了对 MSCAs 和 HTHs 的防护之外，我们还看到与在空间任务中可信地采用 RISC-V 处理器相关的两个主要待解决问题。第一个问题是缺乏可信且稳健的工具链：正如前面几节中已经讨论的那样，已经证明所使用的计算机辅助设计（CAD）工具可能会在设计系统中引入恶意修改。因此，可信工具的可用性以及确保所使用工具可信度的方法至关重要。

第二个主要问题是将第三方知识产权内核（3PIPs）集成到基于 RISC-V 的系统中。专用硬件加速器与主处理器之间的可信交互对于确保整个系统的安全性至关重要。因此，需要研究验证可信行为的方法以及硬件级机制（例如基于硬件的防火墙），以隔离不可信组件并限制其潜在危害。

第五章 总结

RISC-V 微处理器在空间应用中的采用面临着来自硬件安全的多重挑战。尽管其已经具备了一些安全特性，但在应对 MSCAs 和 HTHs 等新型威胁时仍显得力不从心。此外，建立可信的工具链以及确保 3PIPs 的安全集成，是实现 RISC-V 在空间领域可信应用的两个关键问题。这些问题的解决将有助于推动 RISC-V 在空间任务中的广泛应用，并确保其安全性。

面向边缘计算设备的可靠 RISC-V 内核

摘要

将计算从云端迁移到边缘设备（即物联网设备）可以减少延迟和流入网络的数据量。随着开源且可定制的 RISC-V 指令集架构（ISA）的兴起，基于此类 ISA 的内核是物联网家族中多个应用领域的有前途的候选方案，例如汽车、无人机（UAV）、工业自动化、医疗保健、农业等领域，在这些领域中，功耗、实时执行、安全性和可靠性至关重要。在这个新兴的连接 RISC-V 物联网设备的新时代，需要机制来实现可靠和安全的执行，同时满足边缘设备的面积、功耗和计算时间约束。我们提出了三种机制以实现这一目标，即：（i）用于后量子安全启动的信任根模块，（ii）用于防范硬件特洛伊木马和微架构侧信道攻击的硬件检查器，以及（iii）用于实时错误检测和纠正的细粒度双核锁步机制。本文阐述了所提出的机制及其相关动机和影响，并讨论了未来研究方向。

关键词： RISC-V, OpenTitan, 信任根（RoT），安全启动，后量子密码学（PQC），安全，安全检查器，硬件特洛伊木马，微架构侧信道攻击。

I. 引言

为了减少延迟以及流入网络的数据量，将计算从云端迁移到边缘设备（即物联网设备）成为了一种必然趋势。随着开源且可定制的 RISC-V 指令集架构（ISA）的兴起，RISC-V 越来越受到行业的关注[1]。RISC-V 是物联网家族中多个应用领域的有前途的候选方案，例如汽车、无人机（UAV）、工业自动化、医疗保健、农业等领域，在这些领域中，功耗、实时执行、安全性和可靠性至关重要[2][3]。然而，目前大多数关于 RISC-V 的研究主要集中在性能和功耗设计上，往往忽视了可靠性和安全性问题[4]。在这个新兴的连接 RISC-V 物联网设备的新时代，需要机制来实现可靠和安全的执行，同时满足边缘设备的面积、功耗和计算时间约束[5]。

为了实现这一目标，这些机制需要覆盖完整的系统堆栈。整个可靠性和安全性堆栈的基础机制是安全启动，其中执行的代码需要经过身份验证并验证其完整性。系统启动后，系统执行需要受到保护，防止恶意或非恶意注入的故障。这些故障可能会影响指令和数据从内存到处理器的获取，甚至可能影响处理器的执行。

在本工作中，我们提出了三种在系统堆栈不同层面上应用的机制，以实现可靠的 RISC-V 内核。首先，需要通过安全启动方案建立强大的可靠性基础，该方案通过信任根（RoT）模块（如 OpenTitan）与硬件绑定，使用基于公钥密码学（PKC）算法（如 RSA 和 ECC）的数字签名。然而，量子计算技术的出现使得基于整数分解和离散对数的密码学（如 RSA 和 ECC）分别变得可破解[6]。因此，需要适合后量子时代的数据、代码完整性和认证解决方案。然而，由于资源限制，OpenTitan 无法在安全启动流程中支持现有的后量子密码学（PQC）算法，除非进行重大修改。为了克服这一限制，我们提出了 OpenTitan RoT 的增强型安全启动流程，通过修改 OpenTitan 的硬件和软件架构，实现后量子安全。

其次，为了保护从内存到处理器的数据传输过程，需要防止执行未经授权的软件和访问非法内存区域。为此，我们提出了硬件机制来监控指令获取活动、处理的数据以及微处理器的状态，以检测潜在的可疑活动，例如硬件特洛伊木马的激活或微架构侧信道攻击的尝试。

最后，为了应对处理器内部发生的各种硬件故障（例如与制造缺陷、老化或

辐射诱导相关的故障），通常使用基于冗余的机制：在不同的处理器上执行相同的操作集和相同的输入。这种方法非常有效，因为所有处理器同时出现相同故障的概率非常小[7]。然而，当以粗粒度级别应用时，其错误检测和纠正成本较高。因此，我们提出了一个细粒度锁步版本的 RISC-V 处理器，具有快速错误检测和纠正功能。我们使用高级综合（HLS）设计了这种方法。与硬件描述语言（HDL）实现相比，HLS 使设计过程更加简单，因为处理器模型可以轻松修改、扩展和验证[8]。

本文的其余部分安排如下：第 II 节介绍后量子安全启动机制；第 III 节介绍用于防范特洛伊木马和侧信道攻击的硬件安全检查器；第 IV 节讨论用于错误检测和纠正的双核锁步实现；最后，第 V 节总结本文。

II. OpenTitan RoT 上的后量子安全启动

OpenTitan 是一个基于 RISC-V 的开源信任根（RoT），其中一个用例是通过安全启动程序建立信任链的基础。如前所述，OpenTitan 当前的安全启动使用的是不具有后量子安全性的公钥密码学（PKC）方案，因此我们的目标是为 OpenTitan 选择最佳的候选算法。

目前的后量子密码学（PQC）数学解决方案基于哈希、编码、多变量和格理论[9]。考虑到 RISC-V 边缘设备的资源受限环境，PQC 安全启动算法的实现需要满足面积、功耗以及计算时间的要求。第一个基于后量子哈希签名方案的 PQC 硬件安全启动实现已经与硬件和软件 ECDSA 安全启动解决方案进行了比较[10]。基于 LMS 和 SPHINCS+ 算法的 PQC 安全启动签名在认证时间和启动时间以及内存占用方面表现出色[11]，这使得这些算法非常适合资源受限的边缘设备。

在[12]中，对在受限嵌入式设备上实现的基于格的密码学进行了调查，并与传统公钥方案进行了比较。结果显示，随着密钥尺寸的增加，基于格的密码学方案的性能会下降。在 ARM Cortex-M4 微处理器上测试不同基于格的方案的性能时，CRYSTALS-Dilithium 相比其他方案具有最高的吞吐量性能。

CRYSTALS-Dilithium 也是美国国家标准与技术研究院（NIST）后量子密码学标准化项目第三轮的两个签名决赛选手之一[13]，这使其成为 OpenTitan 的理想实现候选方案。然而，这种安全方案意味着复杂的软件实现，由于边缘设备的 ROM 尺寸限制，将其引入物联网设备的初始启动阶段并非易事。

为了克服这一限制并提供抗后量子攻击的启动方案，我们将 OpenTitan 的 RTL（寄存器传输级）扩展以支持 CRYSTALS-Dilithium 加速。我们的设计中使用了 CRYSTALS-Dilithium 硬件原语[14]，以验证启动阶段的签名。

提出的 OpenTitan 的 PQC 安全启动确保只有经过验证的有效载荷才会在系统启动时执行。请注意，本文讨论的安全启动流程不包括 BL0 固件的后续启动阶段以及支持的操作系统内核的启动。启动过程的第一阶段位于启动 ROM 中。ROM 代码在制造过程中被编程到芯片的 ROM 中，且不可更改。它包含一组用于验证存储在闪存中的第一启动阶段（即 ROM 扩展）的公钥。

启动 ROM 代码的主要任务是为执行 ROM 扩展做好准备，同时确保加载的 ROM 扩展被允许在此芯片上执行（ROM 扩展存储在闪存中，并由芯片制造商签名）。图 1 展示了启动代码的两个步骤的概览。

一旦系统上电或复位，启动过程就开始了。执行被限制在 OpenTitan 内部的片上 ROM 中。复位逻辑最初会阻止对其他类型/区域内存的执行。执行进入 ROM 代码，并启用并配置 OpenTitan 的 Ibex 核心的增强型物理内存保护（ePMP）。在这个阶段，只有存储可执行启动代码的 ROM 区域对核心是可访问的。除了保持型 SRAM 外，所有 SRAM 都被清零。从闪存启动信息中加载活动启动槽。闪存启动信息位于闪存的默认银行中。从活动 ROM 扩展槽开始，ROM 代码执行以下步骤：

- 通过检测头部魔数值是否存在，判断活动 ROM 扩展槽是否为空。如果存在，则继续验证 ROM 扩展；如果不存在，则执行启动失败逻辑。
- 如果活动 ROM 扩展槽为空，启动过程读取 ROM 扩展代码并计算其 SHA2-256 摘要。
- 使用 CRYSTALS-Dilithium 硬件原语，通过摘要和存储在 ROM 中的芯片制造商公钥，验证 ROM 扩展清单中的签名。如果验证成功，则继续；如果失败，则执行启动失败逻辑。
- 之后，它执行系统状态测量，并推导出将存储在密钥管理器中的 CreatorRootKey 身份，作为中间状态。
- 接下来，它通过配置适当的 PMP 条目来启用 ROM 扩展的执行。
- 最后，它将执行转移到 ROM 扩展清单中指定的活动槽的入口点。
- 执行跳转到闪存，这是执行就地（XIP）的，并进入 ROM 扩展代码部分。

- ROM 扩展读取启动信息页，以确定它应该启动哪个 BL0/内核映像。
- ROM 扩展读取 BL0 并计算其 SHA2-256 摘要。
- 使用 CRYSTALS-Dilithium 硬件原语，通过摘要和从密钥管理器（KM）派生函数派生的芯片所有者公钥，验证 BL0 清单中的签名。如果验证成功，则继续；如果失败，则执行启动失败逻辑。
- 执行被转移到 BL0/内核映像清单中的入口点。
- 执行进入芯片所有者代码（即 BL0、内核应用程序和数据，这些内容不在本文的讨论范围内）。

我们将提出的方法集成到原始 OpenTitan 流水线中，并在 Xilinx Virtex UltraScale+ VCU118 FPGA 板上进行了评估。FPGA 实现结果如表 I 所示。所提出的 Dilithium 硬件原语占整个 OpenTitan 设计的 33%，因为这种实现是针对高性能进行优化的。为了避免同步问题，我们不得不将其时钟频率降低到 OpenTitan 频率的 1/4。为了降低上述成本，作为未来的工作，我们将在下一个硬件原语设计中权衡性能以换取面积占用的减少，并实现硬件加速器解决方案以卸载 Dilithium 算法的格计算。研究仅将算法中最耗时的部分（即随机生成和多项式乘法操作）卸载到硬件中，并与纯硬件实现进行比较。

III. 硬件安全检查器

RISC-V 架构本身具备多种安全扩展功能，例如特权级别、物理内存保护和密码学功能[15]。然而，这些原生机制并不能保护微处理器免受两大日益严重的威胁：硬件特洛伊木马（Hardware Trojan Horses, HTHs）和微架构侧信道攻击（Microarchitectural Side-Channel Attacks, MSCAs）[16]。

硬件特洛伊木马（HTHs）长期以来一直被认为是一个纯学术问题，因为它们通常表现出的复杂性和危险性都很有限。然而，最近出现了一种新的威胁，使得 HTHs 也成为工业界的关注点：可被软件利用的 HTHs[17]。复杂且高度危险的 HTHs 可能会感染实现微处理器的 IP 核：这些 HTHs 可能允许攻击者执行恶意软件、修改正在运行的软件、获取未经授权的特权或窃取秘密信息[18]。最近在现实世界中发现的一个 HTH 是 Rosenbridge 后门，它存在于一款商业化的 Via Technologies C3 处理器中[19]。这种 HTH 可以通过软件激活和利用，以进入系统的特权模式。尽管已经提出了许多电路级设计阶段的 HTH 检测技术[20]–[22]，但越来越多的研究兴趣集中在系统级技术上，这些技术允许使用不可

信组件构建可信系统[23]、[24]，或者在（部分）不可信的系统上实现可信软件执行[25]、[26]。我们认为，现有的解决方案没有考虑到那些通过使 CPU 执行正常（但意外的）指令而不改变特权模式来改变系统功能的 HTHs。换句话说，这些研究都没有检查微处理器是否正在执行未经授权的软件，以及是否正在访问非法内存地址。

近年来出现的另一种威胁是微架构侧信道攻击（MSCAs），例如 Spectre 和 Meltdown[27]。这些攻击允许攻击者在不需要对被攻击系统进行物理访问的情况下窃取未经授权的信息。事实上，这些攻击仅依赖于对合法硬件功能（例如推测执行和分支预测）的利用，以及对系统在运行敏感应用程序时的时序行为的观察。近年来已经提出了许多针对 MSCAs 的对策[28]。常数时间技术依赖于使执行时间保持恒定以保护秘密信息；编译时技术被提出用于识别和修改那些可能为 MSCAs 打开大门的指令序列；基于操作系统级别的解决方案，例如缓存分区和定期缓存刷新，也被提出。所有这些技术都存在适用范围有限和显著性能下降的问题。最后，许多利用机器学习和硬件性能计数器观察的技术也存在。这些技术能够实现非常高的检测精度，但它们都需要多线程支持，或者需要一个额外的（可信且未被攻击的）核心来运行检测引擎。因此，这些技术适用于高性能计算系统（例如高端服务器），但不适合边缘设备和低端系统（例如智能卡或汽车嵌入式系统），在这些系统中，只有一个核心可用，操作系统要么不存在，要么极其简单。

我们的解决方案（其高级表示如图 2 所示）旨在允许系统集成商通过购买可能不可信的组件来构建可信系统。因此，我们将 IP 提供商视为攻击者；而系统集成员工和代工厂则被视为可信方。我们在架构中引入了硬件安全检查器

（Hardware Security Checkers, HSCs），使得 HSCs 能够监控指令获取活动、处理的数据以及微处理器的状态，以检测潜在的可疑活动，即 HTHs 和/或 MSCAs 的激活。这些 HSCs 在两个阶段工作：在第一个设计阶段，它们学习哪些指令/指令序列是合法的；在随后的运行时监控阶段，它们检查微处理器的活动是否与之前学习到的合法配置文件一致。

1、硬件特洛伊木马的检测

在[29]中，我们提出了一种基于布隆过滤器（Bloom filter）的 HSC，用

于保护微处理器免受主内存中的 HTHs 攻击。在软件安装到指令存储器时, HSC 通过存储有关合法指令及其对应内存地址的信息进行配置。在运行时, HSC 监控微处理器访问的内存地址以及对应的获取指令, 并根据这些信息查询布隆过滤器。如果访问的内存地址和/或对应的获取指令不合法, HSC 将发出警报。在[30]中提出了一个更轻量级的 HSC 版本。

我们将提出的抗 HTH 安全检查器集成到 PULPINO 平台的 RI5CY 版本中, 这是一个小型的 4 级 RISC-V 核心。在 Xilinx Artix XC7A35T 上综合时, RI5CY 需要 15097 个查找表 (LUTs) 和 9881 个触发器 (FFs), 工作频率约为 50MHz, 总功耗为 127mW。我们考虑了以下基准测试: 二分查找 (BinS)、矩阵乘法 (MM)、冒泡排序 (BubS)、快速排序 (QS)、数独求解器 (SS) 和运动检测 (MD)。我们模拟了 10,000 个案例, 其中 HTH 强制从意外内存地址获取恶意指令, 以及 10,000 个案例, 其中 HTH 修改从合法地址读取的获取指令。表 II 报告了两种攻击场景下的误报 (虚假警报) 率和漏报 (未检测到的攻击) 率。可以看出, 我们始终保持着 0% 的误报率, 平均只有大约 2% 的攻击未被检测到, 但仅当 HTH 修改获取的指令时。关于开销, 我们引入了大约 10% 的面积增加, 大约 2% 的功耗增加, 且没有工作频率的降低。

2、微架构侧信道攻击的检测

在[31]中提出了一个与之前介绍的方法类似但用于识别 MSCAs 激活的方法。我们利用 Count-Min Sketch 模型在运行时检测预定义且可重现的可疑指令模式集的执行, 这些模式代表特定的 MSCAs。这些指令模式 (每个感兴趣的攻击一个) 由安全工程师在设计阶段仔细识别, 然后用于配置 HSC。因此, 我们的提议能够并行检查多种微架构攻击, 从而代表了一种灵活且可扩展的安全解决方案。

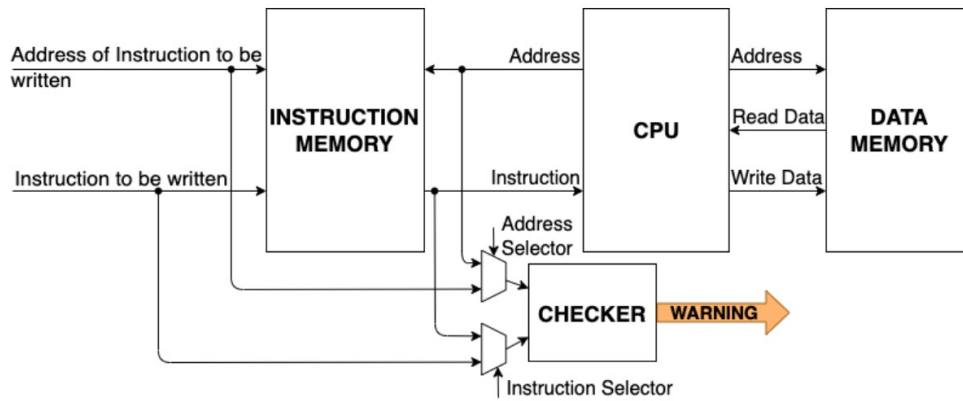


图 1 安全架构

我们将 HSC 集成到 RSD 核心中，这是一个 32 位、推测性乱序、超标量、双获取前端和五发布后端流水线的 RISC-V 核心，具有 16 KB 指令缓存。我们在 Virtex7 FPGA 上实现了目标微处理器，使用了 18334 个查找表（LUTs）、10885 个触发器（FFs）、4512 个 LUTRAM 单元和 17 个 BRAM 单元，工作频率为 57MHz，估计功耗约为 0.926W。我们考虑了 Coremark、Towers、RSort 和 Median 作为受攻击的良性程序，以及 Spectre、Orchestration、Flush+Reload 和 Rowhammer 作为 MSCAs。我们为每个良性程序与攻击程序运行了 10,000 次实验，结果是 100% 的攻击都被检测到。然后，我们测量了引入的 HSC 产生的误报数量：根据 HSC 的配置，误报率从大约 80% 降至 0%。允许我们同时实现 0% 误报率和漏报率的最小 HSC 配置引入了大约 10% 的面积开销，大约 4% 的功耗增加，且没有工作频率的降低。

IV. 锁步双核

处理器冗余可以通过非侵入式或侵入式方式实现，使用同构或异构处理器。非侵入式方法在核心级别引入冗余，而不修改内部处理器微架构，因此通常用于内部微架构细节不可用或难以修改的情况，例如商业现货（Commercial Off-The-Shelf, COTS）处理器。例如，使用两个相同的 MicroBlaze 软核实现同构双核锁步（Dual Core LockStep, DCLS）[32]，其中故障处理器被排除在计算之外，并通过重新配置进行修复，而正确的处理器继续工作。异构非侵入式 DCLS 使用一个 RISC-V 软核和一个 ARM A9 硬核[33]。应用中使用检查点来检测两个核心之间的不匹配，并在检测到不匹配时采用回滚策略。异构方法可能会降低

性能，低速处理器的性能将成为 DCLS 的上限。此外，使用硬核进行锁步需要特定的架构支持，而并非所有处理器都具备这种支持[33]。总体而言，非侵入式方法实现简单，因为无需对微架构进行修改。然而，它们将定义纠正策略（例如通过定期检查点）的责任留给应用层面，可能导致较高的时序开销。

另一方面，侵入式方法修改内部处理器微架构，在实现纠正机制（例如回滚）时更具灵活性。例如，动态自适应冗余架构（Dynamic Adaptive Redundancy Architecture, DARA）是一种应用于 RISC ISA SH-2 处理器的同构侵入式 DCLS 方法，通过回滚实现错误纠正[34]。DARA 添加了额外的硬件来检查锁步核心之间所有流水线阶段寄存器的一致性。然而，DARA 不支持分支指令中发生的故障。另一种同构侵入式方法使用两个虚拟 RISC-V 核心，通过细粒度交错多任务实现 DCLS，以应对共模故障（Common-Mode Failures, CMFs）[35]。其他方法通过扩展流水线寄存器添加错误检测和纠正码，例如 Duckcore 在流水线寄存器中扩展了 RISC-V 核心，使用单错误纠正双错误检测（Single Error Correction Double Error Detection, SECDED）[36]。然而，这种方法可能会因编码和解码时间而增加显著开销，并且不能保证对流水线阶段计算逻辑中故障的保护。其他方法通过在 RISC-V 核心内三重化组件来增强其可靠性。例如，控制和状态寄存器、程序计数器和寄存器文件[37]、触发器（FFs）、查找表（LUTs）、块 RAM（BRAMS）和数字信号处理器（DSPs）[38]，以及算术逻辑单元（ALU）[39]被三重化。总体而言，侵入式方法在实现高效的故障纠正机制方面具有很高的灵活性，但实现起来可能具有挑战性。事实上，现有的侵入式方法基于硬件描述语言（HDL）实现，通常相当复杂。最后，上述方法并未专注于提供有界错误检测和纠正时间。然而，当嵌入式系统被应用于既要求硬实时性又要求可靠执行的领域时，这是关键，因为即使在可能的故障存在的情况下，应用的最坏情况执行时间（Worst-Case Execution Time, WCET）也必须是有界的。

为了解决上述问题，我们提出了一种侵入式的同构 RISC-V 基础的双核锁步实现，提供有界额外时钟周期的瞬态错误检测和纠正。

特别是，在[40]中，我们使用开源的 Comet RV32I RISC-V 架构[8]实现了两种侵入式锁步设计，即部分影子寄存器与回滚（Partial Shadow Register with Rollback, PSRR）和全影子寄存器（Full Shadow Register, FSR），通过高级综合（High-Level Synthesis, HLS）实现。我们使用两个相同的 RISC-V 核心，在每

个时钟周期执行相同的指令。每个流水线阶段将其逻辑计算的结果存储在流水线寄存器中。在每个周期，所提出的机制通过比较两个核心的流水线寄存器来检查执行的一致性，如图 3 所示。如果没有检测到错误，则正常执行。否则，检测到的错误表明故障影响了逻辑，从而产生了错误的结果，或者故障影响了流水线寄存器本身，导致结果中的某一位翻转。在这种情况下，立即进行纠正。在 PSRR 方法中，故障指令被重新获取，并再次通过整个流水线，类似于 DARA[34]。流水线中尚未执行的先前指令保持不变，因为它们的执行未受错误影响；然而，流水线中已经存在的后续指令被丢弃。在 FSR 方法中，如果没有检测到故障，我们创建所有流水线寄存器的备份副本。然后，在检测到故障时，丢弃当前计算的结果，两个核心的流水线寄存器用备份值恢复，最后两个核心重新执行受故障影响的周期。

此外，在[41]中，我们强调了以下关键方面：影响核心的瞬态故障不仅影响应用的功能行为，还会对其时序行为产生显著影响，从而影响 WCET 估计。为了实现这一点，我们利用典型的无故障 WCET 估计方法来实现故障感知，通过考虑核心上发生的瞬态故障的影响。具体来说，我们首先通过广泛的故障注入对目标系统进行脆弱性分析。该分析不仅验证了应用在核心上执行时的功能正确性，还验证了其时序正确性。然后，我们应用一种基于测量的典型 WCET 估计方法来验证故障对 WCET 估计的影响。

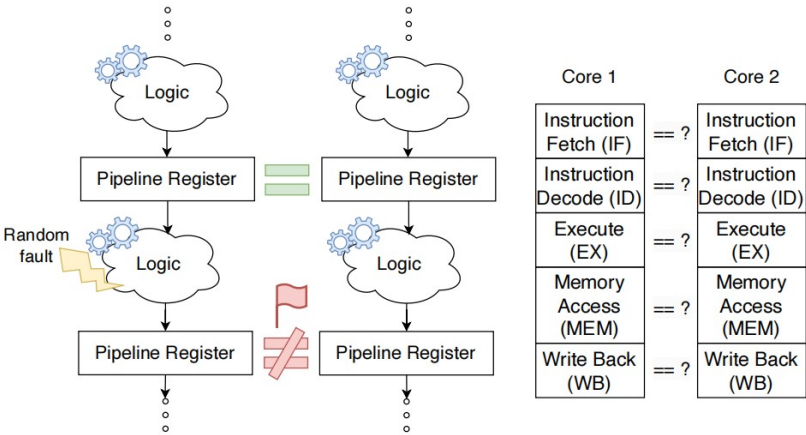


图 2 双核锁步的原理示意图

我们在 TACLeBench（二分查找和素数）、MiBench（快速排序）、AxBench（移动平均）和 Polybench（矩阵乘法）的基准测试中进行了实验。从获得的结果来看，我们观察到在瞬态故障存在的情况下，应用执行时间可能会显著增加，

最高可达 700%，与无故障情况下的应用执行时间相比。此外，与无故障情况下的分布相比，执行时间轨迹的分布也发生了显著变化。

上述观察结果有直接的后果：在故障存在的情况下完成执行所需的时间可能会显著高于无故障的 WCET。因此，现有的方法应该使用看门狗定时器，以限制瞬态故障对应用执行时间的影响，并确保整体调度的安全性。当定时器到期或检测到错误时，应用需要重新执行，根据方法的不同，可能是全部或部分重新执行，从而导致较高的错误检测和纠正时序开销。我们证明了所提出的 DCLS 方法在故障传播并影响执行时间之前立即纠正故障。因此，该方法提供了有界的时序开销（在 FSR 情况下为两个时钟周期），并恢复了 WCET 估计。

V. 讨论与结论

基于 RISC-V 架构的边缘设备正在兴起，这导致了对安全可靠系统的需求。当前的研究提出了三种在 RISC-V 设备上部署的机制，以实现这一目标。随着签名验证算法在后量子时代被证明是不安全的，我们提出了使用 OpenTitan RoT 的物联网设备的后量子安全启动机制。此外，为了检测潜在的可疑活动，我们展示了用于监控指令获取活动、处理的数据以及微处理器状态的硬件机制。最后，我们在双核锁步侵入式 RISC-V 上展示了两种具有有界最坏情况执行时间（WCET）开销的机制，用于检测和从辐射诱导的瞬态故障中恢复。

此外，我们还想强调 RISC-V 开放指令集架构（ISA）能为可靠性研究社区带来的优势。丰富的开源核心实现以及围绕 RISC-V 的庞大社区，使得研究人员能够轻松地实现和测试针对不同可靠性问题的解决方案。然而，我们注意到，目前很少有方法尝试用统一的方法解决不同的挑战，例如，使用相同的硬件同时提供容错和保密性[42]。相反，不同领域的专家（例如可靠性和安全性）通常独立决定他们感兴趣的特定需求，且彼此之间的互动有限。此外，非功能性需求通常被视为可选的附加功能，这增加了相关解决方案的成本。

我们认为，来自不同研究小组且具有不同专业知识的研究人员应该联合起来，将可靠性需求推到未来 RISC-V 核心的主要设计要求之中，这些核心将被用于主流应用。通过这种方式，统一的、高度优化的实现可以蓬勃发展，确保高效且可靠的运行，从而为明天边缘计算设备的 RISC-V 核心提供动力。

参考文献

- [1] [1] C. Palmiero et al., “Design and implementation of a dynamic information flow tracking architecture to secure a risc-v core for iot applications,” in 2018 IEEE High Performance extreme Computing Conference (HPEC). IEEE, 2018, pp. 1 – 7.
- [2] [2] J. Abella et al., “Security, reliability and test aspects of the risc-v ecosystem,” in 2021 IEEE European Test Symposium (ETS), 2021, pp. 1 – 10.
- [3] [3] J. Anders et al., “A survey of recent developments in testability, safety and security of risc-v processors,” in 2023 28th IEEE European Test Symposium (ETS), 2023.
- [4] [4] A. Dorflinger et al., “A comparative survey of open-source application-class risc-v processor implementations,” in Proceedings of the 18th ACM International Conference on Computing Frontiers, ser. CF ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 12 – 20. [Online]. Available: <https://doi.org/10.1145/3457388.3458657>
- [5] [5] Y. H. Hwang, “Iot security & privacy: threats and challenges,” in Proceedings of the 1st ACM workshop on IoT privacy, trust, and security, 2015, pp. 1 – 1.
- [6] [6] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” SIAM review, vol. 41, no. 2, pp. 303 – 332, 1999.
- [7] [7] M. Cui et al., “Fault-tolerant mapping of real-time parallel applications under multiple dvfs schemes,” in IEEE RTAS, 2021, pp. 387 – 399.
- [8] [8] S. Rokicki et al., “What you simulate is what you synthesize: Designing a processor core from c++ specifications,” in 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2019, pp. 1 – 8.

- [9] [9] V. Mavroeidis et al., "The impact of quantum computing on present cryptography," arXiv preprint arXiv:1804.00200, 2018.
- [10] [10] V. B. Kumar et al., "Post-quantum secure boot," in 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2020, pp. 1582 – 1585.
- [11] [11] P. Kampanakis et al., "Post-quantum lms and sphincs+ hash-based signatures for uefi secure boot," Cryptology ePrint Archive, 2021.
- [12] [12] A. Khalid et al., "Lattice-based cryptography for iot in a quantum world: Are we ready?" in 2019 IEEE 8th international workshop on advances in sensors and interfaces (IWASI). IEEE, 2019, pp. 194 – 199.
- [13] [13] J. Zheng et al., "Parallel small polynomial multiplication for dilithium: A faster design and implementation," in Proceedings of the 38th Annual Computer Security Applications Conference, ser. ACSAC ' 22. New York, NY, USA: Association for Computing Machinery, 2022, p. 304 – 317. [Online]. Available: <https://doi.org/10.1145/3564625.3564629>
- [14] [14] L. Beckwith et al., "High-performance hardware implementation of crystals-dilithium," in 2021 International Conference on Field-Programmable Technology (ICFPT). IEEE, 2021, pp. 1 – 10.
- [15] [15] A. Waterman et al., "The risc-v instruction set manual volume ii: Privileged architecture document version 20190608-priv-msu-ratified," RISC-V Foundation, Tech. Rep., 2019.
- [16] [16] L. Cassano et al., "Is risc-v ready for space? a security perspective," in 2022 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), 2022, pp. 1 – 6.
- [17] [17] X. Wang et al., "Software exploitable hardware trojans in embedded processor," in 2012 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), 2012, pp. 55 – 58.
- [18] [18] Y. Jin et al., "Exposing vulnerabilities of untrusted computing platforms," in Proc. Int. Conf. Computer Design, 2012, pp. 131 – 134.
- [19] [19] C. Domas, "Hardware backdoors in x86 cpus," <https://i.blackhat.com/us-18/Thu-August-9/us-18-Domas-God-Mode-Unlocked-Hardware-Backdoors-In-x86-CPU-s-wp.pdf>, 2018.
- [20] [20] X. Chuan et al., "An efficient triggering method of hardware Trojan in AES cryptographic circuit," in Proc. Int. Conf. Integrated Circuits and Microsystems, 2017, pp. 91 – 95.

- [21] [21] Y. Liu et al., "Scca: Side-channel correlation analysis for detecting hardware trojan," in Proc. Int. Conf. Anti-counterfeiting, Security, and Identification, 2017, pp. 196 – 200.
- [22] [22] A. Palumbo et al., "Is your fpga bitstream hardware trojan-free? machine learning can provide an answer," Journal of Systems Architecture, vol. 128, p. 102543, 2022.
- [23] [23] J. Dubeuf et al., "Run-time detection of hardware trojans: The processor protection unit," in 2013 18th IEEE European Test Symposium (ETS), 2013, pp. 1 – 6.
- [24] [24] G. Bloom et al., "Os support for detecting trojan circuit attacks," in 2009 IEEE International Workshop on Hardware-Oriented Security and Trust, 2009, pp. 100 – 103.
- [25] [25] L. Cassano et al., "Deton: Defeating hardware trojan horses in microprocessors through software obfuscation," Journal of Systems Architecture, vol. 129, p. 102592, 2022.
- [26] [26] —, "On the optimization of software obfuscation against hardware trojans in microprocessors," in 2022 25th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS), 2022, pp. 172 – 177.
- [27] [27] A. P. Fournaris et al., "Exploiting hardware vulnerabilities to attack embedded system devices: a survey of potent microarchitectural attacks," Electronics, vol. 6, no. 3, p. 52, 2017.
- [28] [28] Y. Lyu et al., "A survey of side-channel attacks on caches and countermeasures," Journal of Hardware and Systems Security, vol. 2, no. 1, pp. 33 – 50, 2018.
- [29] [29] A. Bolat et al., "A microprocessor protection architecture against hardware trojans in memories," in 2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS). IEEE, 2020, pp. 1 – 6.
- [30] [30] A. Palumbo et al., "A lightweight security checking module to protect microprocessors against hardware trojan horses," in 2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT). IEEE, 2021, pp. 1 – 6.
- [31] [31] K. Arıkan et al., "Processor security: Detect

